

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_634e841e-51b\agent-a349b8f3365e45c3a.jsonl`
- SHA-256: `db920c1252cd071c768253f080296935d4493a0cb90e436f1fa39360b961c122`
- Source modified: `2026-07-06T08:43:56+00:00`
- Imported at: `2026-07-08T16:00:06+00:00`
- project: `wf_634e841e-51b`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-06T08:39:27.318Z)

Reviewing GitHub PR #38 of the repo at C:/Users/avidu/Projects/Telegram (branch feat/track-d-command-service -> main, one commit 22abe72).

It implements ADR 0011 Track D: a REFACTOR that extracts command parsing + config-mutation logic from the

Telethon adapter into a provider-independent CommandService. THE PRIMARY REVIEW RISK IS BEHAVIORAL

EQUIVALENCE: the new code must behave identically to the old inline logic for every command.

Changed files ONLY: src/tg_video_filter/command_service.py (NEW),
src/tg_video_filter/commands.py (refactored
407->~120 line thin Telethon adapter), tests/test_command_service.py (NEW, 20 tests). Diff
+650/-376.
tests/test_commands.py is UNCHANGED and still passes (backwards-compat surface).

To compare against the pre-refactor behaviour, read the OLD file:

```
git -C C:/Users/avidu/Projects/Telegram show main:src/tg_video_filter/commands.py
```

The new logic lives in command_service.py (CommandService.handle + parsers + formatters) and the new

commands.py delegates to it. Key already-checked facts (verify independently, don't just trust):

- regex patterns are byte-identical old vs new; branch order is the same.
- old handler did text.rstrip() then per-branch logic; new handler passes raw text and handle() does text.rstrip().
- old _update_config RELOADED cfg from db each call; new CommandService._update uses the passed current_cfg,
and the new adapter loads current_cfg = db.load_filter_config(...) fresh right before handle(). Assess equivalence.
- both old and new use cfg.model_copy(update=...) which SKIPS pydantic validation (pre-existing, not new).

Ground rules:

- READ actual code (Read/Grep), cite file:line. You MAY run repros with
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe.
- A STRONG equivalence check: feed a battery of command strings through BOTH the new CommandService.handle()
and (on a checkout of main, or by reasoning from the old source) the old handler, and diff
reply_text +
resulting saved FilterConfig. Report ANY divergence (different reply string, different config
mutation,
different validation threshold, different branch taken, different order).
- Authoritative gate ALREADY RUN (don't re-run full suite): ruff check clean; ruff format clean;
pytest
482 passed @ 97.05% (floor 80%, up from 96.52%).
- Report ONLY defects in THIS PR's diff. For a refactor, a BEHAVIORAL DIVERGENCE from the old

code is the

highest-value finding. Also flag: lost functionality, broken re-exports, new bugs in extracted logic.

No praise. Concrete failure scenario per finding. Rank most-severe first. Honest severity.

- Severity: blocker (wrong behaviour users hit / lost command), high (real divergence, narrow trigger),

medium (edge divergence or latent), low (hygiene/style).

DIMENSION: correctness of the extracted CommandService logic (independent of the old code).

Investigate: parse_size_mb (the 'k' branch does max(1, raw//1024) ? is KB handling sane/consistent with old?);

parse_duration_s; the min/max duration cross-validation (max_secs>0 guard when max is None/0); model_copy skips

pydantic validation ? are ALL constraint-violating inputs guarded by explicit pre-checks

(size<1, height<1,

duration bounds)? Can any /set command persist a FilterConfig that violates its own validators (e.g. via a path

that skips the guard)? /show config reloads from db ignoring current_cfg ? intended?

_format_duration/_format_size

edge cases (0, exactly 1024MB, fractional GB). Return findings per schema.

2. assistant (2026-07-06T08:39:30.014Z)

I'll review PR #38 for behavioral equivalence. Let me start by reading both the old and new code.

3. user (2026-07-06T08:39:32.165Z)

```
"""Config commands: /set size, /set length, /show config.
```

These commands are sent to the user's own Saved Messages (like /backfill) and update the persisted ``FilterConfig``. The filter engine is hot-reloaded so the new config takes effect immediately without a restart.

See ``docs/adr/0007-inline-config-commands.md`` for the design rationale.

```
"""
```

```
from __future__ import annotations
```

```
import logging
```

```
import re
```

```
from collections.abc import Callable
```

```
import aiosqlite
```

```
from telethon import TelegramClient
```

```
from telethon.errors import RPCError
```

```
from tg_video_filter import db
```

```
from tg_video_filter.filters import FilterEngine
```

```
from tg_video_filter.models import FilterConfig, User
```

```
log = logging.getLogger(__name__)
```

Key into the shared engine_holder dict. Dict swap is atomic under the GIL for single-threaded asyncio dispatch (see ADR 0007).

```
ENGINE_KEY = "engine"
```

--- regex patterns -----

```
_SIZE_RE = re.compile(r"^/set\s+size\s+(\d+)\s*([kmgT]?b?)$", re.IGNORECASE)
```

Matches both "/set length" and "/set max length" (legacy alias).

Kept separate from `_MIN_LENGTH_RE` for clarity; patterns are mutually exclusive.

```
_LENGTH_RE = re.compile(r"^/set\s+(?:max\s+)?length\s+(\d+)\s*([smh]?)$", re.IGNORECASE)
_MIN_LENGTH_RE = re.compile(r"^/set\s+min\s+length\s+(\d+)\s*([smh]?)$", re.IGNORECASE)
_MIN_RES_RE = re.compile(r"^/set\s+min\s+(?:resolution|res)\s+(\d+)\s*p?$", re.IGNORECASE)
_SHOW_CONFIG_RE = re.compile(r"^/show\s+config$", re.IGNORECASE)
_SHOW_COMMANDS_RE = re.compile(r"^/show\s+commands$", re.IGNORECASE)
_EXCLUDE_RE = re.compile(r"^/set\s+exclude\s+(gif|round|nosound)\s+(on|off)$", re.IGNORECASE)
```

Catch-all: matches any `/set exclude ...` that didn't match the strict pattern, so we can reply with a friendly usage message instead of silence.

```
_EXCLUDE_HELP_RE = re.compile(r"^/set\s+exclude\s+", re.IGNORECASE)
```

Catch-all for malformed `/set min resolution|res` (same pattern as `_EXCLUDE_HELP_RE`)

```
_MIN_RES_HELP_RE = re.compile(r"^/set\s+min\s+(?:resolution|res)\s+", re.IGNORECASE)
```

--- unit parsing -----

```
def parse_size_mb(value_str: str, unit: str) -> int:
    """Parse a size value + unit string into mebibytes.
```

```
>>> parse_size_mb("10", "MB")
10
>>> parse_size_mb("1", "GB")
1024
>>> parse_size_mb("500", "KB")
1
```

```
Note: returns the raw arithmetic result; callers must enforce
`FilterConfig` domain constraints (e.g. `max_size_mb >= 1`).
"""
```

```
value = int(value_str)
unit = unit.lower().rstrip("b") # "MB" ? "m", "KB" ? "k", "GB" ? "g"
multipliers: dict[str, int] = {"": 1, "m": 1, "k": 1, "g": 1024, "t": 1024 * 1024}
if unit in multipliers:
    raw = value * multipliers[unit]
    if unit == "k":
        raw = max(1, raw // 1024)
    return raw
raise ValueError(f"Unknown size unit: {unit}")
```

```
def parse_duration_s(value_str: str, unit: str) -> int:
    """Parse a duration value + unit string into seconds.
```

```
>>> parse_duration_s("15", "m")
900
>>> parse_duration_s("2", "h")
7200
>>> parse_duration_s("90", "s")
90
```

```
Note: returns the raw arithmetic result; callers must enforce
`FilterConfig` domain constraints (e.g. max >= min).
"""
```

```
value = int(value_str)
unit = unit.lower()
multipliers: dict[str, int] = {"": 1, "s": 1, "m": 60, "h": 3600}
if unit in multipliers:
    return value * multipliers[unit]
raise ValueError(f"Unknown time unit: {unit}")
```

--- formatting helpers -----

```
def _format_duration(seconds: int | None) -> str:
    """Format seconds as a human-readable duration string."""
    if seconds is None:
        return "unlimited"
    if seconds == 0:
        return "0s"
    if seconds < 60:
        return f"{seconds}s"
    if seconds < 3600:
        m, s = divmod(seconds, 60)
        return f"{m}m{s}s" if s else f"{m}m"
    h, rem = divmod(seconds, 3600)
    m, s = divmod(rem, 60)
    parts = f"{h}h"
    if m:
        parts += f"{m}m"
    if s:
        parts += f"{s}s"
    return parts

def _format_size(mb: int | None) -> str:
    """Format mebibytes as a human-readable size string."""
    if mb is None:
        return "unlimited"
    if mb == 0:
        return "0MB"
    if mb >= 1024:
        gb = mb / 1024
        if gb == int(gb):
            return f"{int(gb)}GB"
        return f"{gb:.1f}GB"
    return f"{mb}MB"

def format_config(cfg: FilterConfig) -> str:
    """Format a ``FilterConfig`` as a readable Telegram message."""
    lines = [
        "? **Current Filter Configuration**",
        "",
        f"? **Duration:** {_format_duration(cfg.min_duration_s)} - "
        f"{_format_duration(cfg.max_duration_s)}",
        f"? **Min resolution:** {' ' + str(cfg.target_height) + 'p' if cfg.target_height else "disabled"}",
        f"? **Max file size:** {_format_size(cfg.max_size_mb)}",
        f"? **Delivery mode:** `{cfg.delivery_mode.value}`",
        f"? **Delivery target:** `{cfg.delivery_target}`",
        "",
        "? **Content exclusions:**",
        f"    GIF/animated: {'? excluded' if cfg.exclude_gif else '? allowed'}",
        f"    Round videos: {'? excluded' if cfg.exclude_round else '? allowed'}",
        f"    Silent (no audio): {'? excluded' if cfg.exclude_nosound else '? allowed'}",
    ]
    return "\n".join(lines)

def format_commands() -> str:
    """Return a help message listing all available commands."""
    return "\n".join(
        [
            "? **Available Commands**",
            "",
        ]
    )
```

```

    "? **Filter config**",
    "/show config` ? Show current filter settings",
    "/set size <N><unit>` ? Set max file size (e.g. `/set size 10MB`)",
    "    Units: `MB` (default), `GB`, `KB` ? case-insensitive, optional trailing `B`",
    "/set length <N><unit>` ? Set max video duration (e.g. `/set length 15m`)",
    "    or `/set max length <N><unit>` ? same as above",
    "    Units: `s` (default), `m`, `h` ? case-insensitive",
    "/set min length <N><unit>` ? Set min video duration (e.g. `/set min length 5s`)",
    "    Units: `s` (default), `m`, `h` ? case-insensitive",
    "/set min resolution <N>p` ? Set min video height (e.g. `/set min resolution
720p`)",
    "    or `/set min res <N>p` ? same as above",
    "",
    "? **Content exclusions** (all off by default)",
    "/set exclude gif on|off` ? Exclude Telegram GIFs/animated documents",
    "/set exclude round on|off` ? Exclude circular video messages",
    "/set exclude nosound on|off` ? Exclude videos without audio\n"
    "    Note: GIFs are usually silent; `exclude nosound` will catch most GIFs too.",
    "",
    "? **Backfill**",
    "/backfill [N][hdw]` ? Scan last N hours/days/weeks of channel history (default
90d)",
    "/stop` ? Cancel running backfill",
    "",
    "\u2139\u2013 **Help**",
    "/show commands` ? Show this help message",
    ]
)

```

--- command handler factory -----

```

def make_config_command_handler(
    user: User,
    conn: aiosqlite.Connection,
    engine_holder: dict[str, FilterEngine],
) -> Callable:
    """Return an async handler for /set and /show config commands.

    The handler only responds to messages the user sends to themselves
    (Saved Messages). It persists config changes via ``db.save_filter_config``
    and hot-reloads the filter engine through ``engine_holder``.

    ``engine_holder`` is a mutable dict that both the live-message handler and
    the backfill handler read from, so a config change is picked up by all
    components immediately.
    """

    async def on_command(event: object) -> None:
        message = getattr(event, "message", None)
        if message is None:
            return

        # Only respond to messages the user sent to themselves.
        if not getattr(message, "out", False):
            return
        peer = getattr(message, "peer_id", None)
        peer_user_id = getattr(peer, "user_id", None) if peer else None
        if peer_user_id != user.id:
            return

        text: str = getattr(message, "message", "") or ""
        # Strip trailing whitespace so mobile keyboards don't silently

```

```

# reject commands (e.g. "/show config " ? "/show config").
text = text.rstrip()
client: TelegramClient = getattr(event, "client", None) # type: ignore[assignment]

# --- /set size <value><unit> -----
m = _SIZE_RE.match(text)
if m:
    value_str, unit = m.group(1), m.group(2) or ""
    try:
        new_mb = parse_size_mb(value_str, unit)
    except ValueError:
        await _reply(client, message, "? Invalid size format. Use e.g. `/set size
10MB`.")
        return
    if new_mb < 1:
        await _reply(
            client, message, "? Size must be at least 1 MB (`FilterConfig` constraint).")
        )
        return
    cfg = await _update_config(conn, user.id, max_size_mb=new_mb)
    engine_holder[ENGINE_KEY] = FilterEngine(cfg)
    await _reply(
        client,
        message,
        f"? Max file size set to **{_format_size(new_mb)}**."
f"\n\n{format_config(cfg)}",
    )
    return

# --- /set length <value><unit> -----
m = _LENGTH_RE.match(text)
if m:
    value_str, unit = m.group(1), m.group(2) or ""
    try:
        new_secs = parse_duration_s(value_str, unit)
    except ValueError:
        await _reply(
            client, message, "? Invalid duration format. Use e.g. `/set length 15m`."
        )
        return
    # Enforce max_duration_s >= min_duration_s so every video isn't
    # silently rejected (and so the FilterConfig validator passes on
    # next startup).
    current_cfg = await db.load_filter_config(conn, user.id)
    min_secs = current_cfg.min_duration_s or 0
    if new_secs < min_secs:
        await _reply(
            client,
            message,
            f"? Max duration ({_format_duration(new_secs)}) must be "
            f">= min duration ({_format_duration(min_secs)}).",
        )
        return
    cfg = await _update_config(conn, user.id, max_duration_s=new_secs)
    engine_holder[ENGINE_KEY] = FilterEngine(cfg)
    await _reply(
        client,
        message,
        f"? Max duration set to **{_format_duration(new_secs)}**."
f"\n\n{format_config(cfg)}",
    )
    return

# --- /set min length <value><unit> -----
m = _MIN_LENGTH_RE.match(text)

```

```

if m:
    value_str, unit = m.group(1), m.group(2) or ""
    try:
        new_min = parse_duration_s(value_str, unit)
    except ValueError:
        await _reply(
            client,
            message,
            "? Invalid duration format. Use e.g. `/set min length 5s`.",
        )
        return
    # Enforce min_duration_s <= max_duration_s.
    current_cfg = await db.load_filter_config(conn, user.id)
    max_secs = current_cfg.max_duration_s or 0
    if max_secs > 0 and new_min > max_secs:
        await _reply(
            client,
            message,
            f"? Min duration ({_format_duration(new_min)}) must be "
            f"<= max duration ({_format_duration(max_secs)}).",
        )
        return
    cfg = await _update_config(conn, user.id, min_duration_s=new_min)
    engine_holder[ENGINE_KEY] = FilterEngine(cfg)
    await _reply(
        client,
        message,
        f"? Min duration set to **{_format_duration(new_min)}**."
        f"\n\n{format_config(cfg)}",
    )
    return

# --- /set min resolution <N>p -----
m = _MIN_RES_RE.match(text)
if m:
    new_height = int(m.group(1))
    # Pre-validate for a friendly message (FilterConfig also enforces ge=1).
    if new_height < 1:
        await _reply(client, message, "? Resolution must be at least 1p.")
        return
    cfg = await _update_config(conn, user.id, target_height=new_height)
    engine_holder[ENGINE_KEY] = FilterEngine(cfg)
    await _reply(
        client,
        message,
        f"? Min resolution set to **{new_height}p**." f"\n\n{format_config(cfg)}",
    )
    return

# Friendly error for malformed /set min resolution|res.
if _MIN_RES_HELP_RE.match(text):
    await _reply(
        client,
        message,
        "? Usage: `/set min resolution <N>p` (e.g. `/set min resolution 720p`)",
    )
    return

# --- /set exclude <gif|round|nosound> <on|off> -----
m = _EXCLUDE_RE.match(text)
if m:
    what = m.group(1).lower()
    state = m.group(2).lower()
    enabled = state == "on"
    labels = {"gif": "GIF/animated", "round": "round videos", "nosound": "silent

```

```

videos"}

    field_map = {
        "gif": "exclude_gif",
        "round": "exclude_round",
        "nosound": "exclude_nosound",
    }
    cfg = await _update_config(conn, user.id, **{field_map[what]: enabled})
    engine_holder[ENGINE_KEY] = FilterEngine(cfg)
    await _reply(
        client,
        message,
        f"? {labels[what]} exclusion: **{'ON' if enabled else 'OFF'}**."
        f"\n\n{format_config(cfg)}",
    )
    return

# Friendly error for invalid /set exclude (e.g. "gif true", missing on/off).
if _EXCLUDE_HELP_RE.match(text):
    await _reply(
        client,
        message,
        "? Usage: `/set exclude <gif|round|nosound> <on|off>`\n"
        "Example: `/set exclude gif on`",
    )
    return

# --- /show config -----
if _SHOW_CONFIG_RE.match(text):
    cfg = await db.load_filter_config(conn, user.id)
    await _reply(client, message, format_config(cfg))
    return

# --- /show commands -----
if _SHOW_COMMANDS_RE.match(text):
    await _reply(client, message, format_commands())
    return

return on_command

```

--- internal helpers -----

```

async def _update_config(
    conn: aiosqlite.Connection,
    user_id: int,
    max_size_mb: int | None = None,
    max_duration_s: int | None = None,
    min_duration_s: int | None = None,
    target_height: int | None = None,
    exclude_gif: bool | None = None,
    exclude_round: bool | None = None,
    exclude_nosound: bool | None = None,
) -> FilterConfig:
    """Load the current config, apply partial updates, persist, and return."""
    cfg = await db.load_filter_config(conn, user_id)
    if max_size_mb is not None:
        cfg = cfg.model_copy(update={"max_size_mb": max_size_mb})
    if max_duration_s is not None:
        cfg = cfg.model_copy(update={"max_duration_s": max_duration_s})
    if min_duration_s is not None:
        cfg = cfg.model_copy(update={"min_duration_s": min_duration_s})
    if target_height is not None:
        cfg = cfg.model_copy(update={"target_height": target_height})

```



```

if exclude_gif is not None:
    cfg = cfg.model_copy(update={"exclude_gif": exclude_gif})
if exclude_round is not None:
    cfg = cfg.model_copy(update={"exclude_round": exclude_round})
if exclude_nosound is not None:
    cfg = cfg.model_copy(update={"exclude_nosound": exclude_nosound})
await db.save_filter_config(conn, user_id, cfg)
log.info(
    "config_updated user=%s max_size_mb=%s min_duration_s=%s max_duration_s=%s "
    "target_height=%s "
    "exclude_gif=%s exclude_round=%s exclude_nosound=%s",
    user_id,
    cfg.max_size_mb,
    cfg.min_duration_s,
    cfg.max_duration_s,
    cfg.target_height,
    cfg.exclude_gif,
    cfg.exclude_round,
    cfg.exclude_nosound,
)
return cfg

async def _reply(client: TelegramClient, to_message: object, text: str) -> None:
    """Send a reply, logging network errors but not raising.

    Only swallows network/RPC errors so programming bugs (KeyError,
    AttributeError, TypeError) still surface loudly during development.
    """
    try:
        await client.send_message("me", text)
    except (OSError, TimeoutError, RPCError):
        log.warning("config_reply_failed", exc_info=True)

__all__ = [
    "ENGINE_KEY",
    "_EXCLUDE_RE",
    "_LENGTH_RE",
    "_MIN_LENGTH_RE",
    "_MIN_RES_RE",
    "_MIN_RES_HELP_RE",
    "_SHOW_COMMANDS_RE",
    "_SHOW_CONFIG_RE",
    "_SIZE_RE",
    "_format_duration",
    "_format_size",
    "_update_config",
    "format_commands",
    "format_config",
    "make_config_command_handler",
    "parse_duration_s",
    "parse_size_mb",
]

```

4. user (2026-07-06T08:39:32.170Z)

```

1     """Provider-independent command parsing and execution (ADR 0011, Track D).
2
3     Extracts command logic from the Telethon adapter (`commands.py`) so the
4     same ``/set``, ``/show`` commands work identically through a Bot API
5     front-door. The service is pure: strings in, ``CommandResult`` out ? no
6     Telethon objects, no network calls.
7
8     See ``docs/adr/0007-inline-config-commands.md`` for the command model and

```

```

9      ``docs/adr/0008-bot-api-onboarding-front-door.md`` for the Bot API motivation.
10      """
11
12      from __future__ import annotations
13
14      import logging
15      import re
16      from dataclasses import dataclass
17
18      import aiosqlite
19
20      from tg_video_filter import db
21      from tg_video_filter.models import FilterConfig
22
23      log = logging.getLogger(__name__)
24
25      # Key into the shared engine_holder dict (imported by commands.py's adapter).
26      ENGINE_KEY = "engine"
27
28      # --- regex patterns -----
29
30      _SIZE_RE = re.compile(r"^/set\s+size\s+(\d+)\s*([kmgT]?b?)$", re.IGNORECASE)
31      _LENGTH_RE = re.compile(r"^/set\s+(?:max\s+)?length\s+(\d+)\s*([smh]?)$", re.IGNORECASE)
32      _MIN_LENGTH_RE = re.compile(r"^/set\s+min\s+length\s+(\d+)\s*([smh]?)$", re.IGNORECASE)
33      _MIN_RES_RE = re.compile(r"^/set\s+min\s+(?:resolution|res)\s+(\d+)\s*p?$",
re.IGNORECASE)
34      _SHOW_CONFIG_RE = re.compile(r"^/show\s+config$", re.IGNORECASE)
35      _SHOW_COMMANDS_RE = re.compile(r"^/show\s+commands$", re.IGNORECASE)
36      _EXCLUDE_RE = re.compile(r"^/set\s+exclude\s+(gif|round|nosound)\s+(on|off)$",
re.IGNORECASE)
37      _EXCLUDE_HELP_RE = re.compile(r"^/set\s+exclude\s+", re.IGNORECASE)
38      _MIN_RES_HELP_RE = re.compile(r"^/set\s+min\s+(?:resolution|res)\s+", re.IGNORECASE)
39
40
41      # --- unit parsing -----
42
43
44      def parse_size_mb(value_str: str, unit: str) -> int:
45          """Parse a size value + unit string into mebibytes."""
46          value = int(value_str)
47          unit = unit.lower().rstrip("b")
48          multipliers: dict[str, int] = {"": 1, "m": 1, "k": 1, "g": 1024, "t": 1024 * 1024}
49          if unit in multipliers:
50              raw = value * multipliers[unit]
51              if unit == "k":
52                  raw = max(1, raw // 1024)
53              return raw
54          raise ValueError(f"Unknown size unit: {unit}")
55
56
57      def parse_duration_s(value_str: str, unit: str) -> int:
58          """Parse a duration value + unit string into seconds."""
59          value = int(value_str)
60          unit = unit.lower()
61          multipliers: dict[str, int] = {"": 1, "s": 1, "m": 60, "h": 3600}
62          if unit in multipliers:
63              return value * multipliers[unit]
64          raise ValueError(f"Unknown time unit: {unit}")
65
66
67      # --- formatting helpers -----
68
69
70      def _format_duration(seconds: int | None) -> str:
71          if seconds is None:

```

```

72         return "unlimited"
73     if seconds == 0:
74         return "0s"
75     if seconds < 60:
76         return f"{seconds}s"
77     if seconds < 3600:
78         m, s = divmod(seconds, 60)
79         return f"{m}m{s}s" if s else f"{m}m"
80     h, rem = divmod(seconds, 3600)
81     m, s = divmod(rem, 60)
82     parts = f"{h}h"
83     if m:
84         parts += f"{m}m"
85     if s:
86         parts += f"{s}s"
87     return parts
88
89
90 def _format_size(mb: int | None) -> str:
91     if mb is None:
92         return "unlimited"
93     if mb == 0:
94         return "0MB"
95     if mb >= 1024:
96         gb = mb / 1024
97         if gb == int(gb):
98             return f"{int(gb)}GB"
99         return f"{gb:.1f}GB"
100     return f"{mb}MB"
101
102
103 def format_config(cfg: FilterConfig) -> str:
104     """Format a ``FilterConfig`` as a readable message (Telegram markdown)."""
105     lines = [
106         "? **Current Filter Configuration**",
107         "",
108         f"? **Duration:** {_format_duration(cfg.min_duration_s)} -",
109         f"{_format_duration(cfg.max_duration_s)}",
110         f"? **Min resolution:** {'?' + str(cfg.target_height) + 'p' if",
111         f"? **Max file size:** {_format_size(cfg.max_size_mb)}",
112         f"? **Delivery mode:** `{cfg.delivery_mode.value}`",
113         f"? **Delivery target:** `{cfg.delivery_target}`",
114         "",
115         "? **Content exclusions:**",
116         f"    GIF/animated: {'?' excluded' if cfg.exclude_gif else '?' allowed'}",
117         f"    Round videos: {'?' excluded' if cfg.exclude_round else '?' allowed'}",
118         f"    Silent (no audio): {'?' excluded' if cfg.exclude_nosound else '?' allowed'}",
119     ]
120     return "\n".join(lines)
121
122 def format_commands() -> str:
123     """Return a help message listing all available commands."""
124     return "\n".join(
125         [
126             "? **Available Commands**",
127             "",
128             "? **Filter config**",
129             "`/show config` ? Show current filter settings",
130             "`/set size <N><unit>` ? Set max file size (e.g. `/set size 10MB`)",
131             "    Units: `MB` (default), `GB`, `KB` ? case-insensitive, optional trailing",
132             "`B`",
133             "`/set length <N><unit>` ? Set max video duration (e.g. `/set length 15m`)",
134             "    or `/set max length <N><unit>` ? same as above",

```

```

134         "    Units: `s` (default), `m`, `h` ? case-insensitive",
135         "`/set min length <N><unit>` ? Set min video duration (e.g. `/set min length
5s`)",
136         "    Units: `s` (default), `m`, `h` ? case-insensitive",
137         "`/set min resolution <N>p` ? Set min video height (e.g. `/set min
resolution 720p`)",
138         "    or `/set min res <N>p` ? same as above",
139         "",
140         "? **Content exclusions** (all off by default)",
141         "`/set exclude gif on|off` ? Exclude Telegram GIFs/animated documents",
142         "`/set exclude round on|off` ? Exclude circular video messages",
143         "`/set exclude nosound on|off` ? Exclude videos without audio\n"
144         "    Note: GIFs are usually silent; `exclude nosound` will catch most GIFs
too.",
145         "",
146         "? **Backfill**",
147         "`/backfill [N][hdw]` ? Scan last N hours/days/weeks of channel history
(default 90d)",
148         "`/stop` ? Cancel running backfill",
149         "",
150         "\u2139\u2013 **Help**",
151         "`/show commands` ? Show this help message",
152     ]
153 )
154
155
156 # --- command result -----
157
158
159 @dataclass
160 class CommandResult:
161     """The output of executing a command ? provider-independent.
162
163     The adapter (Telethon or Bot API) uses ``reply_text`` to send the response
164     and ``config_changed`` to decide whether to hot-reload the filter engine.
165     """
166
167     reply_text: str = ""
168     config_changed: bool = False
169
170
171 # --- command service -----
172
173
174 class CommandService:
175     """Provider-independent command execution.
176
177     Parses a command string against an account's current config and DB,
178     returning a ``CommandResult``. No Telethon objects involved ? pure string
179     in, effects out. The Telethon adapter (``commands.py``) handles the
180     transport (checking message.out, sending the reply); the Bot API adapter
181     (future) calls ``handle()`` with the ``from.id`` from the update.
182
183     Usage::
184
185         svc = CommandService()
186         result = await svc.handle(user_id, text, conn, current_cfg)
187         if result.config_changed:
188             engine_holder["engine"] = FilterEngine(current_cfg)
189         await send_reply(result.reply_text)
190     """
191
192 # -- public API -----
193
194 async def handle(

```

```

195         self,
196         user_id: int,
197         text: str,
198         conn: aiosqlite.Connection,
199         current_cfg: FilterConfig,
200     ) -> CommandResult:
201         """Parse and execute a command text, returning the reply + change flag."""
202         text = text.rstrip()
203
204         # --- /set size <value><unit> -----
205         m = _SIZE_RE.match(text)
206         if m:
207             value_str, unit = m.group(1), m.group(2) or ""
208             try:
209                 new_mb = parse_size_mb(value_str, unit)
210             except ValueError:
211                 return CommandResult("? Invalid size format. Use e.g. `/set size
10MB`.")
212             if new_mb < 1:
213                 return CommandResult("? Size must be at least 1 MB (`FilterConfig`
constraint).")
214             cfg = await self._update(conn, user_id, current_cfg, max_size_mb=new_mb)
215             return CommandResult(
216                 f"? Max file size set to
**[_format_size(new_mb)]*.\\n\\n{format_config(cfg)}",
217                 config_changed=True,
218             )
219
220         # --- /set length <value><unit> -----
221         m = _LENGTH_RE.match(text)
222         if m:
223             value_str, unit = m.group(1), m.group(2) or ""
224             try:
225                 new_secs = parse_duration_s(value_str, unit)
226             except ValueError:
227                 return CommandResult("? Invalid duration format. Use e.g. `/set length
15m`.")
228             min_secs = current_cfg.min_duration_s or 0
229             if new_secs < min_secs:
230                 return CommandResult(
231                     f"? Max duration ({_format_duration(new_secs)}) must be "
232                     f">= min duration ({_format_duration(min_secs)})."
233                 )
234             cfg = await self._update(conn, user_id, current_cfg,
max_duration_s=new_secs)
235             return CommandResult(
236                 f"? Max duration set to
**[_format_duration(new_secs)]*.\\n\\n{format_config(cfg)}",
237                 config_changed=True,
238             )
239
240         # --- /set min length <value><unit> -----
241         m = _MIN_LENGTH_RE.match(text)
242         if m:
243             value_str, unit = m.group(1), m.group(2) or ""
244             try:
245                 new_min = parse_duration_s(value_str, unit)
246             except ValueError:
247                 return CommandResult("? Invalid duration format. Use e.g. `/set min
length 5s`.")
248             max_secs = current_cfg.max_duration_s or 0
249             if max_secs > 0 and new_min > max_secs:
250                 return CommandResult(
251                     f"? Min duration ({_format_duration(new_min)}) must be "
252                     f"<= max duration ({_format_duration(max_secs)})."

```

```

253         )
254         cfg = await self._update(conn, user_id, current_cfg, min_duration_s=new_min)
255         return CommandResult(
256             f"? Min duration set to
**[_format_duration(new_min)]**.\\n\\n{format_config(cfg)}",
257             config_changed=True,
258         )
259
260     # --- /set min resolution <N>p -----
261     m = _MIN_RES_RE.match(text)
262     if m:
263         new_height = int(m.group(1))
264         if new_height < 1:
265             return CommandResult("? Resolution must be at least 1p.")
266         cfg = await self._update(conn, user_id, current_cfg,
target_height=new_height)
267         return CommandResult(
268             f"? Min resolution set to **{new_height}p**.\\n\\n{format_config(cfg)}",
269             config_changed=True,
270         )
271
272     if _MIN_RES_HELP_RE.match(text):
273         return CommandResult(
274             "? Usage: `/set min resolution <N>p` (e.g. `/set min resolution 720p`)"
275         )
276
277     # --- /set exclude <gif|round|nosound> <on|off> -----
278     m = _EXCLUDE_RE.match(text)
279     if m:
280         what = m.group(1).lower()
281         state = m.group(2).lower()
282         enabled = state == "on"
283         labels = {"gif": "GIF/animated", "round": "round videos", "nosound": "silent
videos"}
284         field_map = {
285             "gif": "exclude_gif",
286             "round": "exclude_round",
287             "nosound": "exclude_nosound",
288         }
289         cfg = await self._update(conn, user_id, current_cfg, **{field_map[what]:
enabled})
290         return CommandResult(
291             f"? {labels[what]} exclusion: **{'ON' if enabled else 'OFF'}**."
292             f"\\n\\n{format_config(cfg)}",
293             config_changed=True,
294         )
295
296     if _EXCLUDE_HELP_RE.match(text):
297         return CommandResult(
298             "? Usage: `/set exclude <gif|round|nosound> <on|off>`\\n"
299             "Example: `/set exclude gif on`"
300         )
301
302     # --- /show config -----
303     if _SHOW_CONFIG_RE.match(text):
304         cfg = await db.load_filter_config(conn, user_id)
305         return CommandResult(format_config(cfg))
306
307     # --- /show commands -----
308     if _SHOW_COMMANDS_RE.match(text):
309         return CommandResult(format_commands())
310
311     # Not a recognised command ? return empty result (noise).
312     return CommandResult()
313

```

```

314 # -- internal -----
315
316 async def _update(
317     self,
318     conn: aiosqlite.Connection,
319     user_id: int,
320     current: FilterConfig,
321     max_size_mb: int | None = None,
322     max_duration_s: int | None = None,
323     min_duration_s: int | None = None,
324     target_height: int | None = None,
325     exclude_gif: bool | None = None,
326     exclude_round: bool | None = None,
327     exclude_nosound: bool | None = None,
328 ) -> FilterConfig:
329     """Apply partial updates to the account's config, persist, and return."""
330     updates: dict[str, object] = {}
331     if max_size_mb is not None:
332         updates["max_size_mb"] = max_size_mb
333     if max_duration_s is not None:
334         updates["max_duration_s"] = max_duration_s
335     if min_duration_s is not None:
336         updates["min_duration_s"] = min_duration_s
337     if target_height is not None:
338         updates["target_height"] = target_height
339     if exclude_gif is not None:
340         updates["exclude_gif"] = exclude_gif
341     if exclude_round is not None:
342         updates["exclude_round"] = exclude_round
343     if exclude_nosound is not None:
344         updates["exclude_nosound"] = exclude_nosound
345     cfg = current.model_copy(update=updates) if updates else current
346     await db.save_filter_config(conn, user_id, cfg)
347     log.info(
348         "config_updated user=%s max_size_mb=%s min_duration_s=%s max_duration_s=%s "
349         "target_height=%s exclude_gif=%s exclude_round=%s exclude_nosound=%s",
350         user_id,
351         cfg.max_size_mb,
352         cfg.min_duration_s,
353         cfg.max_duration_s,
354         cfg.target_height,
355         cfg.exclude_gif,
356         cfg.exclude_round,
357         cfg.exclude_nosound,
358     )
359     return cfg
360
361 __all__ = [
362     "ENGINE_KEY",
363     "CommandResult",
364     "CommandService",
365     "_format_duration",
366     "_format_size",
367     "format_config",
368     "format_commands",
369     "parse_duration_s",
370     "parse_size_mb",
371 ]
372
373

```

5. user (2026-07-06T08:39:32.252Z)

```

1     """Config commands ? Telethon adapter over the provider-independent `CommandService`.
2

```

```

3      This module is a thin Telethon adapter. It handles Telethon-specific transport
4      (checking `message.out`, `peer_id`, sending replies) and delegates command
5      parsing and config mutation to `command_service.CommandService` (ADR 0011, Track D).
6
7      Re-exports regex patterns and unit-parsers for backwards-compatibility with
8      existing tests (`tests/test_commands.py`).
9
10     See `docs/adr/0007-inline-config-commands.md` for the command model and
11     `docs/adr/0008-bot-api-onboarding-front-door.md` for the Bot API front-door.
12     """
13
14     from __future__ import annotations
15
16     import logging
17     from collections.abc import Callable
18
19     import aiosqlite
20     from telethon import TelegramClient
21     from telethon.errors import RPCError
22
23     from tg_video_filter import db
24     from tg_video_filter.command_service import ENGINE_KEY, CommandResult, CommandService
25     from tg_video_filter.filters import FilterEngine
26     from tg_video_filter.models import FilterConfig, User
27
28     log = logging.getLogger(__name__)
29
30     from tg_video_filter.command_service import ( # noqa: E402
31         _EXCLUDE_HELP_RE,
32         _EXCLUDE_RE,
33         _LENGTH_RE,
34         _MIN_LENGTH_RE,
35         _MIN_RES_HELP_RE,
36         _MIN_RES_RE,
37         _SHOW_COMMANDS_RE,
38         _SHOW_CONFIG_RE,
39         _SIZE_RE,
40         _format_duration,
41         _format_size,
42         format_commands,
43         format_config,
44         parse_duration_s,
45         parse_size_mb,
46     )
47
48
49     def make_config_command_handler(
50         user: User,
51         conn: aiosqlite.Connection,
52         engine_holder: dict[str, FilterEngine],
53     ) -> Callable:
54         svc = CommandService()
55
56         async def on_command(event: object) -> None:
57             message = getattr(event, "message", None)
58             if message is None:
59                 return
60             if not getattr(message, "out", False):
61                 return
62             peer = getattr(message, "peer_id", None)
63             peer_user_id = getattr(peer, "user_id", None) if peer else None
64             if peer_user_id != user.id:
65                 return
66
67             text: str = getattr(message, "message", "") or ""

```



```

68         client: TelegramClient = getattr(event, "client", None) # type:
ignore[assignment]
69
70         current_cfg = await db.load_filter_config(conn, user.id)
71         result: CommandResult = await svc.handle(user.id, text, conn, current_cfg)
72
73         if result.config_changed:
74             new_cfg = await db.load_filter_config(conn, user.id)
75             engine_holder[ENGINE_KEY] = FilterEngine(new_cfg)
76
77         if result.reply_text:
78             await _reply(client, message, result.reply_text)
79
80     return on_command
81
82
83 # --- backwards-compat helpers (test_commands.py references) -----
84
85
86 async def _update_config(
87     conn: aisqlite.Connection,
88     user_id: int,
89     max_size_mb: int | None = None,
90     max_duration_s: int | None = None,
91     min_duration_s: int | None = None,
92     target_height: int | None = None,
93     exclude_gif: bool | None = None,
94     exclude_round: bool | None = None,
95     exclude_nosound: bool | None = None,
96 ) -> FilterConfig:
97     """Deprecated wrapper ? delegates to ``CommandService._update``.
98
99     Kept for backwards-compatibility with ``tests/test_commands.py`` which
100     imports ``_update_config`` from here. New code should call
101     :meth:`CommandService.handle` which returns a :class:`CommandResult`.
102     """
103     svc = CommandService()
104     cfg = await db.load_filter_config(conn, user_id)
105     return await svc._update(
106         conn,
107         user_id,
108         cfg,
109         max_size_mb=max_size_mb,
110         max_duration_s=max_duration_s,
111         min_duration_s=min_duration_s,
112         target_height=target_height,
113         exclude_gif=exclude_gif,
114         exclude_round=exclude_round,
115         exclude_nosound=exclude_nosound,
116     )
117
118
119 async def _reply(client: TelegramClient, to_message: object, text: str) -> None:
120     try:
121         await client.send_message("me", text)
122     except (OSError, TimeoutError, RPCError):
123         log.warning("config_reply_failed", exc_info=True)
124
125
126 __all__ = [
127     "ENGINE_KEY",
128     "_EXCLUDE_RE",
129     "_LENGTH_RE",
130     "_MIN_LENGTH_RE",
131     "_MIN_RES_RE",

```

```

132     "_SHOW_CONFIG_RE",
133     "_SHOW_COMMANDS_RE",
134     "_EXCLUDE_HELP_RE",
135     "_MIN_RES_HELP_RE",
136     "_SIZE_RE",
137     "_format_duration",
138     "_format_size",
139     "_update_config",
140     "format_commands",
141     "format_config",
142     "make_config_command_handler",
143     "parse_duration_s",
144     "parse_size_mb",
145 ]
146

```

6. assistant (2026-07-06T08:39:34.811Z)

Now let me look at the FilterConfig model to understand its validators, then run a behavioral equivalence battery.

7. user (2026-07-06T08:39:36.910Z)

(Bash completed with no output)

8. user (2026-07-06T08:39:37.609Z)

```

14:from pydantic import BaseModel, ConfigDict, Field, field_validator, model_validator
299:class FilterConfig(BaseModel):
308:    min_duration_s: int | None = Field(default=5, ge=0)
309:    max_duration_s: int | None = Field(default=7200, ge=0)
311:    # Resolution: a video passes if its height >= target_height.
313:    target_height: int | None = Field(default=1080, ge=1)
316:    max_size_mb: int | None = Field(default=2048, ge=1)
354:    @field_validator("delivery_target", mode="before")
369:    @model_validator(mode="after")
371:        """Ensure min_duration_s <= max_duration_s when both are set."""
373:        self.min_duration_s is not None
374:        and self.max_duration_s is not None
375:        and self.min_duration_s > self.max_duration_s
378:        f"min_duration_s ({self.min_duration_s}) must be <= max_duration_s
({self.max_duration_s})"

```

9. user (2026-07-06T08:39:40.743Z)

```

299    class FilterConfig(BaseModel):
300        """Per-user filter configuration.
301
302        v1 rules: duration bounds, minimum height (resolution), and max file size.
303        Stored in the DB as a JSON blob (`filter_config.config_json`) so adding a
304        new field later is backwards-compatible.
305        """
306
307        # Duration bounds in seconds. None means unbounded on that side.
308        min_duration_s: int | None = Field(default=5, ge=0)
309        max_duration_s: int | None = Field(default=7200, ge=0)
310
311        # Resolution: a video passes if its height >= target_height.
312        # None disables the resolution rule.
313        target_height: int | None = Field(default=1080, ge=1)
314
315        # Max file size in mebibytes. None disables the size rule.
316        max_size_mb: int | None = Field(default=2048, ge=1)
317

```

```

318 # Content-type exclusions. GIFs and round videos are excluded by default
319 # (their duration/dimensions are usually incidental to real video filters);
320 # silent videos are off by default since some legit clips have no audio.
321 # Existing users' stored configs are unaffected ? the DB JSON blob overrides
322 # these defaults, so this only changes the behaviour for fresh installs.
323 exclude_gif: bool = Field(
324     default=True,
325     description="If True, exclude Telegram GIFs (DocumentAttributeAnimated)",
326 )
327 exclude_round: bool = Field(
328     default=True,
329     description="If True, exclude circular video messages (round_message=True)",
330 )
331 exclude_nosound: bool = Field(
332     default=False,
333     description="If True, exclude videos without an audio track (nosound=True)",
334 )
335
336 # How matched videos are delivered: forward the video, send a link, or both.
337 # Stored in the same JSON blob, so adding this field is backwards-compatible
338 # (existing rows without it default to FORWARD).
339 delivery_mode: DeliveryMode = Field(default=DeliveryMode.FORWARD)
340
341 # Where matched videos / link messages are sent. Accepts:
342 # - "me" (default): Saved Messages ? exact v1.1 behaviour.
343 # - "@username": a public channel the account is a member of.
344 # - "-100..." or a bare int-as-string: a numeric Telegram chat id
345 # (private channels without a username).
346 # The string is passed verbatim to Telethon's forward_messages / send_message.
347 # See ADR 0005 for rationale and access requirements.
348 delivery_target: str = Field(
349     default="me",
350     min_length=1,
351     description="Telethon entity to deliver to: 'me', '@username', or a chat id",
352 )
353
354 @field_validator("delivery_target", mode="before")
355 @classmethod
356 def _coerce_delivery_target(cls, v: object) -> str:
357     """Accept int chat ids (e.g. -1001234567890) and coerce to str.
358
359     Telethon accepts both str and int as an entity, but storing as str keeps
360     the JSON blob uniform and lets '@username' / 'me' / numeric ids share
361     one field type.
362     """
363     if v is None:
364         return "me"
365     if isinstance(v, int):
366         return str(v)
367     return str(v)
368
369 @model_validator(mode="after")
370 def _check_duration_bounds(self) -> FilterConfig:
371     """Ensure min_duration_s <= max_duration_s when both are set."""
372     if (
373         self.min_duration_s is not None
374         and self.max_duration_s is not None
375         and self.min_duration_s > self.max_duration_s
376     ):
377         raise ValueError(
378             f"min_duration_s ({self.min_duration_s}) must be <= max_duration_s "
379             f"({self.max_duration_s})"
380         )
381     return self

```

```
382
383     class VideoRecord(BaseModel):
```

10. assistant (2026-07-06T08:39:52.868Z)

Now I have the full picture. Let me analyze the key equivalence concern raised in the prompt about `\_update\_config` reloading vs. using `current\_cfg`. The critical difference:

- **Old**: each `\_update\_config` call does `cfg = await db.load\_filter\_config(conn, user\_id)` fresh, then applies updates.
- **New**: `\_update` uses the passed `current` cfg; adapter loads `current\_cfg` once before `handle()`.

Within a single command this is equivalent (only one `\_update` call per command). Let me now run the behavioral battery to compare old vs new directly.

11. user (2026-07-06T08:39:53.986Z)

```
feat(track-d-command-service
22abe72 feat(commands): extract CommandService ? provider-independent command execution (ADR
0011, Track D)
```

12. assistant (2026-07-06T08:39:57.999Z)

I'll write an equivalence harness that reconstructs the old handler logic as a pure function and compares it against the new `CommandService.handle` across a battery of inputs.

13. user (2026-07-06T08:40:01.384Z)

saved

14. assistant (2026-07-06T08:40:38.571Z)

"""Behavioral equivalence harness: old inline handler vs new CommandService.

We emulate the DB with an in-memory FilterConfig store. Both paths share the same store so we compare (reply_text, resulting saved config) per input.

```
"""
import asyncio
import sys
sys.path.insert(0, "C:/Users/avidu/Projects/Telegram/src")

from tg_video_filter.models import FilterConfig
from tg_video_filter import command_service as newmod
from tg_video_filter.command_service import CommandService
```

Fake db module shared by both paths -- but old handler calls db.* and new calls db.*
We monkeypatch tg_video_filter.db.load_filter_config / save_filter_config.

```
import tg_video_filter.db as dbmod
```

```
class Store:
    def __init__(self, cfg):
        self.cfg = cfg
        self.saved = []
```

STORE = None

```
async def fake_load(conn, user_id):
    return STORE.cfg
```

```
async def fake_save(conn, user_id, cfg):
    STORE.cfg = cfg
    STORE.saved.append(cfg)
```

```
dbmod.load_filter_config = fake_load
dbmod.save_filter_config = fake_save
```

---- OLD handler logic reconstructed as pure fn (from main:commands.py) -----

We replicate the branch bodies but capture reply + saved cfg instead of network.

```
from tg_video_filter.command_service import (
    _SIZE_RE, _LENGTH_RE, _MIN_LENGTH_RE, _MIN_RES_RE, _SHOW_CONFIG_RE,
    _SHOW_COMMANDS_RE, _EXCLUDE_RE, _EXCLUDE_HELP_RE, _MIN_RES_HELP_RE,
    parse_size_mb, parse_duration_s, _format_size, _format_duration,
    format_config, format_commands,
)

async def old_update_config(conn, user_id, **kw):
    # OLD: reload from db each call
    cfg = await dbmod.load_filter_config(conn, user_id)
    for k, v in kw.items():
        if v is not None:
            cfg = cfg.model_copy(update={k: v})
    await dbmod.save_filter_config(conn, user_id, cfg)
    return cfg

async def old_handle(user_id, text, conn):
    """Returns (reply_text_or_None, config_changed_bool)."""
    text = text.rstrip()
    replies = []
    def reply(t): replies.append(t)
    changed = False

    m = _SIZE_RE.match(text)
    if m:
        value_str, unit = m.group(1), m.group(2) or ""
        try:
            new_mb = parse_size_mb(value_str, unit)
        except ValueError:
            reply("? Invalid size format. Use e.g. `/set size 10MB`.")
            return replies, changed
        if new_mb < 1:
            reply("? Size must be at least 1 MB (`FilterConfig` constraint).")
            return replies, changed
        cfg = await old_update_config(conn, user_id, max_size_mb=new_mb)
        changed = True
        reply(f"? Max file size set to **{_format_size(new_mb)}**." f"\n\n{format_config(cfg)}")
        return replies, changed

    m = _LENGTH_RE.match(text)
    if m:
        value_str, unit = m.group(1), m.group(2) or ""
        try:
            new_secs = parse_duration_s(value_str, unit)
        except ValueError:
            reply("? Invalid duration format. Use e.g. `/set length 15m`.")
            return replies, changed
        current_cfg = await dbmod.load_filter_config(conn, user_id)
        min_secs = current_cfg.min_duration_s or 0
        if new_secs < min_secs:
            reply(f"? Max duration ({_format_duration(new_secs)}) must be "
                  f">= min duration ({_format_duration(min_secs)}).")
            return replies, changed
        cfg = await old_update_config(conn, user_id, max_duration_s=new_secs)
        changed = True
        reply(f"? Max duration set to **{_format_duration(new_secs)}**."
              f"\n\n{format_config(cfg)}")
        return replies, changed
```

```

m = _MIN_LENGTH_RE.match(text)
if m:
    value_str, unit = m.group(1), m.group(2) or ""
    try:
        new_min = parse_duration_s(value_str, unit)
    except ValueError:
        reply(f"? Invalid duration format. Use e.g. `/set min length 5s`.")
        return replies, changed
    current_cfg = await dbmod.load_filter_config(conn, user_id)
    max_secs = current_cfg.max_duration_s or 0
    if max_secs > 0 and new_min > max_secs:
        reply(f"? Min duration ({_format_duration(new_min)}) must be "
              f"<= max duration ({_format_duration(max_secs)}).")
        return replies, changed
    cfg = await old_update_config(conn, user_id, min_duration_s=new_min)
    changed = True
    reply(f"? Min duration set to **{_format_duration(new_min)}**."
          f"\n\n{format_config(cfg)}")
    return replies, changed

m = _MIN_RES_RE.match(text)
if m:
    new_height = int(m.group(1))
    if new_height < 1:
        reply(f"? Resolution must be at least 1p.")
        return replies, changed
    cfg = await old_update_config(conn, user_id, target_height=new_height)
    changed = True
    reply(f"? Min resolution set to **{new_height}p**." f"\n\n{format_config(cfg)}")
    return replies, changed

if _MIN_RES_HELP_RE.match(text):
    reply(f"? Usage: `/set min resolution <N>p` (e.g. `/set min resolution 720p`)"
          f"\n\n{format_config(cfg)}")
    return replies, changed

m = _EXCLUDE_RE.match(text)
if m:
    what = m.group(1).lower(); state = m.group(2).lower(); enabled = state == "on"
    labels = {"gif": "GIF/animated", "round": "round videos", "nosound": "silent videos"}
    field_map = {"gif": "exclude_gif", "round": "exclude_round", "nosound":
"exclude_nosound"}
    cfg = await old_update_config(conn, user_id, **{field_map[what]: enabled})
    changed = True
    reply(f"? {labels[what]} exclusion: **{'ON' if enabled else 'OFF'}**."
          f"\n\n{format_config(cfg)}")
    return replies, changed

if _EXCLUDE_HELP_RE.match(text):
    reply(f"? Usage: `/set exclude <gif|round|nosound> <on|off>`\nExample: `/set exclude gif
on`")
    return replies, changed

if _SHOW_CONFIG_RE.match(text):
    cfg = await dbmod.load_filter_config(conn, user_id)
    reply(format_config(cfg))
    return replies, changed

if _SHOW_COMMANDS_RE.match(text):
    reply(format_commands())
    return replies, changed

return replies, changed # no reply

```

```

async def run():

```

```

global STORE
svc = CommandService()
inputs = [
    "/set size 10MB", "/set size 10", "/set size 1GB", "/set size 500KB",
    "/set size 500k", "/set size 0MB", "/set size 0KB", "/set size 1023KB",
    "/set size 1024KB", "/set size 2048KB", "/set size 2T", "/set size 1t",
    "/set size 3000", "/set size 1536",
    "/set length 15m", "/set length 0", "/set length 2h", "/set max length 1h",
    "/set length 3s", "/set length 90",
    "/set min length 5s", "/set min length 0", "/set min length 10m",
    "/set min length 8000",
    "/set min resolution 720p", "/set min resolution 0p", "/set min res 480",
    "/set min resolution abc",
    "/set exclude gif on", "/set exclude round off", "/set exclude nosound on",
    "/set exclude gif true", "/set exclude foo on",
    "/show config", "/show commands", "/show config ", " /show config",
    "/SET SIZE 10mb", "/set size 10mb",
    "not a command", "", "/backfill",
    "/set length 7200", "/set length 7201",
    "/set min length 7200", "/set min length 7201",
]
diffs = []
for text in inputs:
    # Fresh identical store for each path
    base = FilterConfig()
    # OLD
    STORE = Store(base.model_copy())
    old_replies, old_changed = await old_handle(1, text, None)
    old_cfg = STORE.cfg
    # NEW
    STORE = Store(base.model_copy())
    current_cfg = await dbmod.load_filter_config(None, 1)
    res = await svc.handle(1, text, None, current_cfg)
    new_cfg = STORE.cfg

    old_reply = old_replies[0] if old_replies else None
    new_reply = res.reply_text if res.reply_text else None
    old_dump = old_cfg.model_dump()
    new_dump = new_cfg.model_dump()

    problems = []
    if old_reply != new_reply:
        problems.append(f"REPLY differs:\n OLD={old_reply!r}\n NEW={new_reply!r}")
    if old_dump != new_dump:
        problems.append(f"CONFIG differs:\n OLD={old_dump}\n NEW={new_dump}")
    if old_changed != res.config_changed:
        problems.append(f"CHANGED flag: OLD={old_changed} NEW={res.config_changed}")
    if problems:
        diffs.append((text, problems))

if not diffs:
    print("NO DIVERGENCE across", len(inputs), "inputs")
else:
    print(f"DIVERGENCES ({len(diffs)}):")
    for text, probs in diffs:
        print(f"\n=== input={text!r} ===")
        for p in probs:
            print(p)

```

```

asyncio.run(run())

```

15. user (2026-07-06T08:40:38.572Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

16. assistant (2026-07-06T08:40:43.597Z)

The file doesn't exist yet; the error is spurious protection. Let me use a different filename.

17. user (2026-07-06T08:41:11.999Z)

NO DIVERGENCE across 46 inputs

18. assistant (2026-07-06T08:41:20.282Z)

No divergence across 46 inputs, including the KB rounding, GB, T, edge boundaries, cross-validation, and whitespace cases. The refactor is behaviorally equivalent for single commands.

Now let me examine the correctness dimension independently ? can any `/set` persist a config that violates its own validators (bypassing `model\_copy`)? The key candidate: `/set length` with a value below the current `min\_duration\_s` is guarded, and `/set min length` above `max` is guarded. But let me check the guard asymmetry carefully around the `max\_secs > 0` guard, and whether `\_show config` reload difference matters.

19. user (2026-07-06T08:41:34.131Z)

Exit code 1

Traceback (most recent call last):

```
File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\validators.py", line 41, in <module>
    asyncio.run(run())
    ~~~~~^~~~~~
File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line 195, in run
    return runner.run(main)
    ~~~~~^~~~~~
File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line 118, in run
    return self._loop.run_until_complete(task)
    ~~~~~^~~~~~
File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line 725, in run_until_complete
    return future.result()
    ~~~~~^~~~~~
File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\validators.py", line 21, in run
    ("min=5,max=0; /set min length 100", FilterConfig(min_duration_s=5, max_duration_s=0), "/set min length 100"),
    ~~~~~^~~~~~
File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\pydantic\main.py", line 263, in __init__
    validated_self = self.__pydantic_validator__.validate_python(data, self_instance=self)
pydantic_core._pydantic_core.ValidationError: 1 validation error for FilterConfig
  Value error, min_duration_s (5) must be <= max_duration_s (0) [type=value_error, input_value={'min_duration_s': 5, 'max_duration_s': 0}, input_type=dict]
  For further information visit https://errors.pydantic.dev/2.13/v/value_error
```

20. assistant (2026-07-06T08:41:36.974Z)

That scenario constructs an already-invalid config directly. Let me remove it and focus on valid starting configs that a `/set` could push into invalidity.

21. user (2026-07-06T08:41:49.621Z)

```
min=None,max=None; /set min length 100
reply='? Min duration set to **1m40s**.\n\n? **Current Filter Configuration**\n\n'
saved min=100 max=None changed=True -> VALID

min=None,max=0; /set min length 100
```



```
reply='? Min duration set to **1m40s**.\n\n? **Current Filter Configuration**\n\n'
saved min=100 max=0 changed=True -> *** INVALID PERSISTED ***: Value error, min_duration_s
(100) must be <= max_duration_s (0) [type=value_error, input_value={'min_duration_s': 100,
'...'delivery_target': 'me'}, input_type=dict]
```

```
min=0,max=None; /set length 0
```

```
reply='? Max duration set to **0s**.\n\n? **Current Filter Configuration**\n\n?? '
saved min=0 max=0 changed=True -> VALID
```

```
min=None,max=10; /set length 5
```

```
reply='? Max duration set to **5s**.\n\n? **Current Filter Configuration**\n\n?? '
saved min=None max=5 changed=True -> VALID
```

```
min=100,max=None; /set length 50
```

```
reply='? Max duration (50s) must be >= min duration (1m40s).'
saved min=100 max=None changed=False -> VALID
```

```
min=5,max=None; /set min length 100 (max None)
```

```
reply='? Min duration set to **1m40s**.\n\n? **Current Filter Configuration**\n\n'
saved min=100 max=None changed=True -> VALID
```

```
min=None,max=7200; /set min length 10000
```

```
reply='? Min duration (2h46m40s) must be <= max duration (2h).'
saved min=None max=7200 changed=False -> VALID
```

22. assistant (2026-07-06T08:41:54.517Z)

Found a latent invalid-persist path ? but it's **pre-existing**, not introduced by this PR. Let me verify the old code has the identical ``max_secs > 0`` guard (it does, from the old source I read: ``max_secs = current_cfg.max_duration_s or 0`` then ``if max_secs > 0 and new_min > max_secs``). So the ``max=0`` case (``max_secs > 0`` is False ? guard skipped) exists identically in the old code. This is NOT a regression from this PR.

Let me confirm the old code path is byte-identical for that guard, and confirm this is pre-existing.

23. user (2026-07-06T08:41:56.225Z)

```
297:         max_secs = current_cfg.max_duration_s or 0
298:         if max_secs > 0 and new_min > max_secs:
303:             f"<= max duration ({_format_duration(max_secs)})".,
```

24. assistant (2026-07-06T08:42:03.150Z)

Confirmed identical ``max_secs > 0`` guard in old code (lines 297-298). So the ``max=0`` invalid-persist path is pre-existing and out of scope for this PR's diff.

Now let me check the one genuine behavioral difference the prompt flagged: ``/show config`` reloads from db ignoring ``current_cfg``. In both old and new, ``/show config`` calls ``db.load_filter_config`` fresh. But note a subtle new-adapter concern: the adapter loads ``current_cfg`` **always** (even for ``/show config`` and non-commands), then ``handle`` reloads again for ``/show config``. That's a redundant read but not a behavioral difference. Let me verify there's no double-save or engine-reload divergence, and check the ``config_changed`` ? reload path in the adapter.

The adapter (commands.py lines 70-78): loads ``current_cfg``, calls ``handle``, and if ``config_changed``, does a **second** ``db.load_filter_config`` to build the engine. The old code built the engine from the ``cfg`` returned by ``_update_config`` directly. Let me verify these are equivalent ? they should be since ``_update`` just saved that same `cfg`.

25. user (2026-07-06T08:42:22.311Z)

```
Exit code 1
after /set size 50MB:
```

```

    saved cfg max_size_mb = 50
    engine in holder: True
Traceback (most recent call last):
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\adapter_check.py", line 71, in <module>
    asyncio.run(run())
    ~~~~~^
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line 195, in run
    return runner.run(main)
    ~~~~~^
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line 118, in run
    return self._loop.run_until_complete(task)
    ~~~~~^
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line 725, in run_until_complete
    return future.result()
    ~~~~~^
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\adapter_check.py", line 43, in run
    print("  engine.config.max_size_mb =", eng.config.max_size_mb)
    ~~~~~^
AttributeError: 'FilterEngine' object has no attribute 'config'

```

26. assistant (2026-07-06T08:42:25.638Z)

Minor attribute name issue; the important results are already visible: `/set size 50MB` saved cfg max_size_mb=50 and engine set. Let me fix the attribute and finish the transport checks.

27. user (2026-07-06T08:42:27.309Z)

grep: src/tg_video_filter/filters.py: No such file or directory

28. user (2026-07-06T08:42:35.410Z)

```
['cfg', 'rules']
```

29. assistant (2026-07-06T08:42:37.391Z)

It's `cfg`. Let me rerun the (now fixed) adapter check.

30. user (2026-07-06T08:42:40.370Z)

```

after /set size 50MB:
  saved cfg max_size_mb = 2048
  engine in holder: True
  engine.config.max_size_mb = 2048
  loads= 2 saves= 1
  replies= ['? Max file size set to **50MB**.\n\n? **Cu']

```

```

after /show config:
  engine set: False saves= 0
  reply has 99MB: True

```

```

after non-command 'hello world':
  replies: [] engine set: False

```

```

after not-out /set size: replies: [] saves: 0
after wrong-peer /set size: replies: [] saves: 0

```

31. assistant (2026-07-06T08:42:48.291Z)

Interesting ? the sed didn't apply because I ran it on a `.py` that was already being edited;

but the output shows the real behavior. Note: `saved cfg max\_size\_mb = 2048` and `engine.config.max\_size\_mb = 2048` ? this is because my fake `fake\_save` mutates STORE.cfg but the reply correctly says 50MB. Wait, that's a bug in my harness: `fake\_load` returns `STORE.cfg` and `fake\_save` sets it, but the second `load` after save returns the saved cfg. Let me look ? reply says 50MB (correct), engine says 2048. That means the second reload (`new\_cfg = await db.load\_filter\_config`) returned a stale value.

Actually this reveals my harness `Store` reset issue, not a code bug ? but it's worth verifying the adapter's double-load returns the fresh saved config in a realistic setting. The reply text (from `handle`'s in-memory `cfg`) says 50MB, but the engine (from the adapter's separate reload) showed 2048. Let me investigate this carefully ? it could be a real issue if load/save aren't consistent.

32. user (2026-07-06T08:42:59.283Z)

```
saved max_size_mb: 50
engine cfg max_size_mb: 50
reply: ? Max file size set to **50MB**.
```

```
? **Current
loads: 2 saves: 1
OK: engine reload matches saved config
```

33. assistant (2026-07-06T08:43:05.683Z)

With a realistic single-store DB, the adapter's double-load returns the freshly-saved config (max_size_mb=50) and the engine matches. My earlier harness's `Store` reset caused the phantom 2048. The adapter is correct. The transport guards (not-out, wrong-peer, non-command ? no reply, no save) all behave identically to the old code.

Now the one remaining functional-difference to weigh: the new adapter calls `db.load\_filter\_config` **unconditionally** at line 70 for every message the user sends to themselves ? including plain non-command messages and `/show config`. The old code only hit the DB when a branch matched. This is an extra DB read per self-message but not a behavioral divergence (no user-visible effect). Worth noting as low-severity at most.

Let me also verify the re-export surface is complete ? that `tests/test\_commands.py` imports still resolve.

34. user (2026-07-06T08:43:07.778Z)

```
17:from tg_video_filter.commands import (
---imports used---
```

35. user (2026-07-06T08:43:11.096Z)

```
from tg_video_filter.commands import (
    _EXCLUDE_RE,
    _LENGTH_RE,
    _MIN_LENGTH_RE,
    _MIN_RES_HELP_RE,
    _MIN_RES_RE,
    _SHOW_COMMANDS_RE,
    _SHOW_CONFIG_RE,
    _SIZE_RE,
    ENGINE_KEY,
    _format_duration,
    _format_size,
    _update_config,
    format_commands,
    format_config,
    make_config_command_handler,
    parse_duration_s,
    parse_size_mb,
```

```
)
from tg_video_filter.filters import FilterEngine
from tg_video_filter.models import DeliveryMode, FilterConfig, User
```

--- parse_size_mb -----

```
@pytest.mark.parametrize(
    ("value", "unit", "expected"),
    [
        ("10", "MB", 10),
```

36. assistant (2026-07-06T08:43:18.064Z)

All those names are re-exported by the new `commands.py` (`\_EXCLUDE\_HELP\_RE` is imported at line 31 too but not used by the test list; every name in the test import list is present). Note `\_EXCLUDE\_HELP\_RE` is in the new `commands.py \_\_all\_\_` but the test doesn't import it ? fine. Let me confirm the import resolves and no name is missing.

37. user (2026-07-06T08:43:21.638Z)

all imports resolve OK

38. assistant (2026-07-06T08:43:29.497Z)

All re-exports resolve. Let me do one final targeted check on the KB-branch and `\_format\_size`/`\_format\_duration` edge cases explicitly called out in the prompt, to be thorough on the correctness dimension (independent of old code).

39. user (2026-07-06T08:43:34.398Z)

```
parse_size_mb(1,KB) = 1
parse_size_mb(500,KB) = 1
parse_size_mb(1023,KB) = 1
parse_size_mb(1024,KB) = 1
parse_size_mb(1536,KB) = 1
parse_size_mb(2048,KB) = 2
parse_size_mb(3000,KB) = 2
parse_size_mb(2,T)= 2097152 (2*1024*1024)
--- _format_size ---
None -> unlimited
0 -> 0MB
1 -> 1MB
1023 -> 1023MB
1024 -> 1GB
1536 -> 1.5GB
2048 -> 2GB
3072 -> 3GB
1500 -> 1.5GB
--- _format_duration ---
None -> unlimited
0 -> 0s
1 -> 1s
59 -> 59s
60 -> 1m
61 -> 1m1s
3600 -> 1h
3661 -> 1h1m1s
7200 -> 2h
90 -> 1m30s
3599 -> 59m59s
```

40. assistant (2026-07-06T08:43:44.549Z)

All formatter/parser edge cases behave sanely and are byte-identical to the old implementations (I confirmed the source is identical). The KB branch ``max(1, raw//1024)`` floors sub-1MB values to 1 and truncates (1536KB?1MB, 2048KB?2MB), consistent with the old code.

Summary of my investigation:

- **Behavioral equivalence battery**: 46 command strings run through both the reconstructed old handler and new ``CommandService.handle``, comparing reply text, saved config, and change flag. **Zero divergence.**
- **Adapter transport guards** (not-out, wrong-peer, non-command, ``/show config` no-save`): identical behavior.
- **Adapter double-load** (load current_cfg, then reload for engine after save): returns fresh saved config; engine matches.
- **Validator-bypass probe**: the only path that persists a self-invalid config (``min > max`` when ``max_duration_s == 0``, because the ``max_secs > 0`` guard skips) is **pre-existing** ? byte-identical guard in old code (main:commands.py:297-298). Not introduced by this PR.
- **Re-export surface**: all names imported by the unchanged ``tests/test_commands.py`` resolve.
- **Parsers/formatters**: KB flooring, GB fractional, boundary cases all sane and identical to old.

The refactor is behaviorally equivalent. I found no blocker/high/medium defects introduced by this diff. The only notes are low-severity hygiene observations.

41. user (2026-07-06T08:43:56.197Z)

Structured output provided successfully